

FlashAttention & the Online Softmax, Worked Out by Hand

HOW A STREAMING ALGORITHM REPRODUCES SOFTMAX EXACTLY — WITH A PROOF AND A HAND-CHECK

What this document does. It explains the single mathematical trick that makes FlashAttention possible: the *online (streaming) softmax*. We maintain three running statistics — a running max m , a running denominator ℓ , and a running output O — and update them block by block as tiles of keys arrive. We then *prove* that after the last block the result is identical to a one-shot softmax over the whole row, and confirm it on a worked example with 4 keys split into 2 blocks (✓, agreement to 2×10^{-16}). The payoff is $O(N)$ memory instead of $O(N^2)$: the $N \times N$ score tensor of Topic 1 is never materialized. Every figure comes from Appendix A (m03_t02_flashattention_online_softmax_engine.py).

The one idea. Standard softmax seems to require the whole row at once, because it subtracts the row max for stability. The online softmax shows you can carry a running max and *retroactively rescale* everything computed so far when a larger value appears. That one rescaling identity lets attention run in SRAM-sized tiles — exactly once over HBM — and is why 128K context windows became practical in 2022.

CONCEPTS COVERED, IN ORDER

why tiling attention is blocked by softmax; the three running statistics m, ℓ, O ; the block-update rules; a two-block worked example with a hand-check; a proof that streaming equals one-shot; the $O(N)$ vs $O(N^2)$ memory win and the “two passes, not six” HBM accounting.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The obstacle: softmax wants the whole row	2
2	Step 1 — The three running statistics	2
3	Step 2 — The block-update rules	2
4	Step 3 — A two-block worked example	3
5	Step 4 — Proof that streaming equals one-shot	4
6	Step 5 — The memory win: $O(N)$ not $O(N^2)$	4
7	The argument, at a glance	5
	7.1 Equation cheat-sheet	5
	Appendix A — Reference implementation	5

1 The obstacle: softmax wants the whole row

Recall the stable softmax from Topic 1: for a row of scores $s_1, \dots, s_N$,

$$P_i = \frac{\exp(s_i - m)}{\sum_k \exp(s_k - m)}, \quad m = \max_k s_k.$$

To compute *any* P_i you need m (the max over the whole row) and $\ell = \sum_k \exp(s_k - m)$ (the sum over the whole row). If you stream the keys in tiles, each tile sees only a slice of the row — so neither m nor ℓ is known until the last tile.

Intuition

This is the entire reason naive attention materializes the $N \times N$ score matrix: it computes all scores first so it can take the row max and row sum, *then* does softmax. FlashAttention wants to process keys in small tiles that fit in SRAM, so it needs a softmax that can be computed incrementally, fixing up its earlier work as new larger scores appear. That is the online softmax.

2 Step 1 — The three running statistics

As we stream over key blocks $j = 1, 2, \dots$, maintain for each query row:

Symbol	Meaning	Init
m	running max of all scores seen so far	$-\infty$
ℓ	running denominator $\sum \exp(s - m)$, rescaled to current m	0
O	running output (probability-weighted value sum), rescaled to current m	0

Block j contributes its own local statistics: local max $m_j = \max_{k \in j} s_k$, local denom $\ell_j = \sum_{k \in j} \exp(s_k - m_j)$, and local output $O_j = \sum_{k \in j} \exp(s_k - m_j) V_k$ (unnormalized, relative to its own local max).

3 Step 2 — The block-update rules

When block j arrives, fold it into the running state. First the new max, then rescale the old state down to it and the new block down to it:

Online softmax update — the core of FlashAttention

$$\begin{aligned} m_{\text{new}} &= \max(m_{\text{old}}, m_j) \\ \ell_{\text{new}} &= \ell_{\text{old}} e^{m_{\text{old}} - m_{\text{new}}} + \ell_j e^{m_j - m_{\text{new}}} \\ O_{\text{new}} &= \frac{O_{\text{old}} \ell_{\text{old}} e^{m_{\text{old}} - m_{\text{new}}} + O_j e^{m_j - m_{\text{new}}}}{\ell_{\text{new}}} \end{aligned}$$

The two correction factors $e^{m_{\text{old}} - m_{\text{new}}}$ and $e^{m_j - m_{\text{new}}}$ are always ≤ 1 : they re-express every previously accumulated term *as if* it had been computed against the new, larger max. After the last block, O is the exact attention output.

Intuition

The implementation (Appendix A) carries an *unnormalized* accumulator $\text{acc} = \sum \exp(s - m) V$ and divides by ℓ only once at the very end — algebraically identical to rescaling O each step, but with one division instead of N . Both forms appear below; they agree exactly.

4 Step 3 — A two-block worked example

One query attends over 4 keys with pre-scaled scores $s = (1, 3, 2, 0)$ and value rows

$$V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 2 & 2 \end{bmatrix},$$

split into block 1 = keys $\{1, 2\}$ and block 2 = keys $\{3, 4\}$. We track the unnormalized accumulator acc and denominator ℓ .

Block 1: keys $\{1, 2\}$, scores $(1, 3)$

Local max $m_1 = 3$. Local exponentials $e^{1-3} = 0.1353$, $e^{3-3} = 1$, so $\ell_1 = 1.1353$ and

$$O_1 = 0.1353 (1, 0) + 1 (0, 1) = (0.1353, 1.0000).$$

This is the first block, so the running state becomes $m = 3$, $\ell = 1.1353$, $\text{acc} = (0.1353, 1.0000)$.

Block 2: keys $\{3, 4\}$, scores $(2, 0)$

Local max $m_2 = 2$. Exponentials $e^{2-2} = 1$, $e^{0-2} = 0.1353$, so $\ell_2 = 1.1353$ and

$$O_2 = 1 (1, 1) + 0.1353 (2, 2) = (1.2707, 1.2707).$$

New max $m_{\text{new}} = \max(3, 2) = 3$. Correction factors: old state $e^{3-3} = 1$, new block $e^{2-3} = 0.3679$. Update:

$$\begin{aligned} \ell &= 1.1353 \cdot 1 + 1.1353 \cdot 0.3679 = 1.5530, \\ \text{acc} &= (0.1353, 1.0000) \cdot 1 + (1.2707, 1.2707) \cdot 0.3679 = (0.6028, 1.4675). \end{aligned}$$

Normalize once at the end:

$$O = \frac{\text{acc}}{\ell} = \frac{(0.6028, 1.4675)}{1.5530} = (0.3881, 0.9449). \quad \checkmark$$

Reference: one-shot softmax over all four keys

Full-row max $m = 3$; $\exp(s - 3) = (0.1353, 1, 0.3679, 0.0498)$; $\ell = 1.5530$; $P = (0.0871, 0.6439, 0.2369, 0.0321)$; and

$$O_{\text{ref}} = P V = (0.3881, 0.9449).$$

The streaming output matches to 2.22×10^{-16} — machine precision. $\checkmark$ Note the streaming denominator $\ell = 1.5530$ already equals the one-shot ℓ .

5 Step 4 — Proof that streaming equals one-shot

The example is not a coincidence. Here is the general argument.

The streaming state is an exact partial softmax

Claim. After processing blocks $1, \dots, J$ (covering key set $\mathcal{S}$), the running state satisfies, with $M = \max_{k \in \mathcal{S}} s_k$,

$$m = M, \quad \ell = \sum_{k \in \mathcal{S}} e^{s_k - M}, \quad \text{acc} = \sum_{k \in \mathcal{S}} e^{s_k - M} V_k.$$

Base case ($J=1$). A single block: $m = m_1 = \max_{k \in \text{blk } 1} s_k$, $\ell = \sum e^{s_k - m_1}$, $\text{acc} = \sum e^{s_k - m_1} V_k$. Holds by definition.

Inductive step. Assume it holds for $\mathcal{S}$ (old state m, ℓ, acc). Add block $J+1$ with key set $\mathcal{B}$, local max m_j , local $\ell_j = \sum_{k \in \mathcal{B}} e^{s_k - m_j}$, $O_j = \sum_{k \in \mathcal{B}} e^{s_k - m_j} V_k$. Let $m' = \max(m, m_j) = \max_{k \in \mathcal{S} \cup \mathcal{B}} s_k$. Then

$$\ell e^{m - m'} = \sum_{k \in \mathcal{S}} e^{s_k - m} e^{m - m'} = \sum_{k \in \mathcal{S}} e^{s_k - m'}, \quad \ell_j e^{m_j - m'} = \sum_{k \in \mathcal{B}} e^{s_k - m'},$$

because the rescaling exponents *telescope*: $e^{s_k - m} e^{m - m'} = e^{s_k - m'}$. Their sum is exactly $\sum_{k \in \mathcal{S} \cup \mathcal{B}} e^{s_k - m'}$, which is the claim for ℓ_{new} . The identical telescoping applies term-by-term to acc (each value V_k is carried along its own exponent). So the claim holds for $\mathcal{S} \cup \mathcal{B}$. ■

Why it is exact, not approximate

The whole proof is the telescoping identity $e^{s_k - m} e^{m - m'} = e^{s_k - m'}$. Rescaling by $e^{m - m'}$ does not approximate anything — it re-bases an exponential onto a new reference max with zero error. After the final block $m = M$ (the true row max) and ℓ, acc are the true row sums, so $O = \text{acc}/\ell$ is the exact softmax-weighted output. Streaming attention is *bit-identical* (to floating point) with the one-shot version, not a lossy approximation. The script's 50-trial randomized stress test confirms a worst-case error of 6.7×10^{-16} .

6 Step 5 — The memory win: $O(N)$ not $O(N^2)$

The running state per query row is just (m, ℓ, O) — size $O(d)$, independent of N . The $N \times N$ score matrix is never formed.

Memory at Nexus scale ($N = 6200$, 64 heads, FP16)

One materialized score matrix per head (Topic 1):

$$N^2 \cdot 2 \text{ B} = 6200^2 \cdot 2 = 76.9 \text{ MB} \quad [O(N^2)].$$

FlashAttention's per-query running state is $O(d)$; at $d = 128$, FP16, the carried (m, ℓ, O) is on the order of 256 B per row [$O(N)$ total]. The streaming state is roughly 48× smaller *per head* than a single score matrix — and crucially does not grow quadratically with context. ✓

NAIVE • 6 HBM trips

Materialize S, P ($N \times N$) in HBM.
 read $Q, K \rightarrow$ write $S \rightarrow$ read $S \rightarrow$
 write $P \rightarrow$ read $P, V \rightarrow$ write O .
 Memory: $O(N^2)$ per head.

FLASH • 2 HBM passes

Tile Q, K, V into SRAM; online softmax.
 read Q, K, V once $\rightarrow$ compute in SRAM
 $\rightarrow$
 write O once. S, P never touch HBM.
 Memory: $O(N)$ running state.

Intuition

FlashAttention fuses the six round trips of Topic 1 into a single streaming pass: load a Q tile into SRAM, stream K/V tiles through it while running the online-softmax update, and write the finished O tile out once. The $N \times N$ scratch lives and dies inside SRAM, one tile at a time. Same math, same result — 3–5 $\times$ faster on prefill and $O(N)$ memory, which is what unlocked long context across the industry.

7 The argument, at a glance

#	Step	Result
1	Obstacle	stable softmax needs the row max $\Rightarrow$ blocks tiling
2	State	carry m (max), ℓ (denom), O (output)
3	Update	rescale old + new block by $e^{m_{\bullet} - m_{\text{new}}}$
4	Example	2 blocks $\rightarrow O = (0.3881, 0.9449)$ ✓
5	Proof	telescoping $e^{s-m} e^{m-m'} = e^{s-m'} \Rightarrow$ exact
6	Memory	$O(N)$ state, 2 HBM passes vs $O(N^2)$, 6 trips

7.1 Equation cheat-sheet

New max	$m_{\text{new}} = \max(m_{\text{old}}, m_j)$
Denominator	$\ell_{\text{new}} = \ell_{\text{old}} e^{m_{\text{old}} - m_{\text{new}}} + \ell_j e^{m_j - m_{\text{new}}}$
Output	$O_{\text{new}} = (O_{\text{old}} \ell_{\text{old}} e^{m_{\text{old}} - m_{\text{new}}} + O_j e^{m_j - m_{\text{new}}}) / \ell_{\text{new}}$
Local stats	$m_j, \ell_j = \sum e^{s-m_j}, O_j = \sum e^{s-m_j} V$
Key identity	$e^{s-m} e^{m-m'} = e^{s-m'}$ (telescoping, exact)

Appendix A — Reference implementation

Every number above is produced by `m03_t02_flashattention_online_softmax_engine.py`. It runs the two-block example, checks it against the one-shot softmax, reports the memory comparison, and stress-tests 50 random blockings.

```

1 """
2 Reference implementation for: Module 03 - Topic 2
3 "FlashAttention & the Online (Streaming) Softmax, Worked Out by Hand."
4
5 Demonstrates that the streaming online-softmax update produces output that is
6 bit-for-bit identical (to floating-point rounding) to the one-shot softmax, while
7 using  $O(d)$  running state instead of  $O(N)$  score storage.
8
9 Every number printed here is transcribed (rounded) into the worked-by-hand PDF.
10 """
11

```

```

12 import numpy as np
13
14 np.set_printoptions(precision=4, suppress=True)
15
16 # -----
17 # 0. One query attending over 4 keys, split into two blocks of 2.
18 #    d = 2 so the value vectors are easy to read.
19 # -----
20 # Pre-scaled scores (already QK^T / sqrt(d_k)) for the single query row:
21 s = np.array([1.0, 3.0, 2.0, 0.0])      # logits over 4 keys
22 V = np.array([[1.0, 0.0],
23               [0.0, 1.0],
24               [1.0, 1.0],
25               [2.0, 2.0]])              # value rows, d = 2
26
27 print("== 0. Setup: 1 query, 4 keys, d=2 ==")
28 print("scores s =", s)
29 print("V =\n", V)
30
31 # -----
32 # 1. Reference: ONE-SHOT softmax over the full row, then weighted sum.
33 # -----
34 m_full = s.max()
35 e_full = np.exp(s - m_full)
36 l_full = e_full.sum()
37 p_full = e_full / l_full
38 O_full = p_full @ V
39 print("\n== 1. One-shot (reference) ==")
40 print(f"max m      = {m_full}")
41 print(f"exp(s-m)    = {e_full}")
42 print(f"denom l      = {l_full:.6f}")
43 print(f"P          = {p_full}")
44 print(f"O_ref = P@V   = {O_full}")
45
46 # -----
47 # 2. STREAMING: process block 1 = keys[0:2], then block 2 = keys[2:4].
48 #    State carried between blocks: running max m, running denom l,
49 #    running UNNORMALIZED output acc = sum_j exp(s_j - m) * V_j.
50 #    (We normalize O = acc / l only at the very end -- the standard
51 #    FlashAttention bookkeeping.)
52 # -----
53 def block_stats(s_blk, V_blk):
54     mj = s_blk.max()
55     ej = np.exp(s_blk - mj)
56     lj = ej.sum()
57     Oj = ej @ V_blk      # unnormalized, relative to local max mj
58     return mj, lj, Oj
59
60 print("\n== 2. Streaming over 2 blocks ==")
61 m = -np.inf
62 l = 0.0
63 acc = np.zeros(V.shape[1])    # running unnormalized output
64
65 blocks = [(s[0:2], V[0:2]), (s[2:4], V[2:4])]
66 for j, (s_blk, V_blk) in enumerate(blocks, start=1):
67     mj, lj, Oj = block_stats(s_blk, V_blk)
68     m_new = max(m, mj)
69     # rescale old state to the new max, add the new block (rescaled to new max)
70     alpha = np.exp(m - m_new) if np.isfinite(m) else 0.0    # old-state correction
71     beta  = np.exp(mj - m_new)                             # new-block correction
72     l     = l * alpha + lj * beta
73     acc   = acc * alpha + Oj * beta
74     m     = m_new

```

```

75     print(f"  block {j}: local m_j={mj:.4f} l_j={lj:.4f} O_j={Oj}")
76     print(f"          after update -> m={m:.4f} l={l:.6f} acc={acc}")
77
78     O_stream = acc / l
79     print(f"\nO_stream = acc / l = {O_stream}")
80
81     # -----
82     # 3. Equality check: streaming == one-shot.
83     # -----
84     print("\n== 3. Equality check ==")
85     print(f"O_ref      = {O_full}")
86     print(f"O_stream = {O_stream}")
87     print(f"max abs diff = {np.max(np.abs(O_full - O_stream)):.2e}")
88     print(f"identical (allclose)? {np.allclose(O_full, O_stream)}")
89     print(f"denominators equal? l_stream={l:.6f} vs l_ref={l_full:.6f} -> {np.isclose(l, l_full)}")
90
91     # -----
92     # 4. Memory argument: O(N) streaming state vs O(N^2) materialized scores.
93     #   Nexus scale: N=6200, 64 heads, FP16.
94     # -----
95     print("\n== 4. Memory: O(N) running state vs O(N^2) scores ==")
96     N = 6200
97     H = 64
98     d = 128
99     dtype_b = 2
100    score_NN = N * N * dtype_b          # one N x N score matrix per head
101    flash_row = d * dtype_b             # running (m,l,0) is O(d) per query row
102    print(f"one N x N score matrix (per head) = {score_NN/1e6:.1f} MB   [O(N^2)]")
103    print(f"flash per-query running state ~ O(d) = {flash_row} B       [O(N)] total over rows")
104    print(f"flash reads Q,K,V once and writes O once: 2 HBM passes, not 6.")
105    print(f"ratio (scores vs flash state, per head): {score_NN/(N*flash_row):.0f}x less memory")
106
107    # -----
108    # 5. Randomized stress test: 50 random blockings still match one-shot.
109    # -----
110    print("\n== 5. Randomized stress test (50 trials) ==")
111    rng = np.random.default_rng(7)
112    worst = 0.0
113    for _ in range(50):
114        Nk = rng.integers(5, 40)
115        dd = rng.integers(2, 8)
116        sr = rng.standard_normal(Nk) * 5.0      # large spread -> stress numerics
117        Vr = rng.standard_normal((Nk, dd))
118        # one-shot
119        mr = sr.max(); er = np.exp(sr - mr); Oref = (er / er.sum()) @ Vr
120        # streaming with a random block size
121        bs = int(rng.integers(1, Nk))
122        mm, ll, ac = -np.inf, 0.0, np.zeros(dd)
123        for st in range(0, Nk, bs):
124            sb, Vb = sr[st:st+bs], Vr[st:st+bs]
125            mj = sb.max(); ej = np.exp(sb - mj); lj = ej.sum(); Oj = ej @ Vb
126            mn = max(mm, mj)
127            a = np.exp(mm - mn) if np.isfinite(mm) else 0.0
128            b = np.exp(mj - mn)
129            ll = ll * a + lj * b
130            ac = ac * a + Oj * b
131            mm = mn
132        worst = max(worst, np.max(np.abs(ac / ll - Oref)))
133    print(f"worst-case abs error over 50 random blockings = {worst:.2e} (machine-precision)")

```